

TRABAJO PRÁCTICO N° 1 - Año 2026 - 2° Semestre

Problema 1 - Ecualización local de histograma

La técnica de ecualización de histograma puede extenderse para realizar un análisis local de la imagen, dando lugar a la **ecualización local de histograma**. El procedimiento consiste en definir una ventana cuadrada o rectangular de tamaño $M \times N$ y desplazar su centro de píxel en píxel a lo largo de la imagen. En cada posición, se calcula el histograma correspondiente a los píxeles contenidos dentro de la ventana y, mediante el mismo, se obtiene la transformación local de la ecualización del histograma.

Tal transformación se utiliza para determinar el nuevo nivel de intensidad correspondiente al píxel ubicado en el centro de la ventana. Luego, la ventana se desplaza un píxel y el procedimiento se repite hasta recorrer la totalidad de la imagen.

Esta técnica resulta particularmente útil cuando una imagen contiene diferentes zonas cuyos detalles presentan niveles de intensidad similares a los del fondo local. En estos casos, una ecualización global de histograma puede no producir buenos resultados, debido a que el histograma se calcula considerando la totalidad de los píxeles de la imagen y, por lo tanto, se pierde la información local.

El algoritmo a desarrollar debe resolver los siguientes puntos:

- Desarrolle una **función para implementar la ecualización local de histograma**, que reciba como parámetros de entrada la imagen a procesar y el tamaño de la ventana de procesamiento ($M \times N$ con $M, N \in \mathbb{Z}^+$).
- Utilice la **función desarrollada en el punto anterior para analizar la imagen** de la *figura 1* (archivo *Imagen_con_objetos_ocultos.tiff*) e informe cuáles son los detalles ocultos presentes en las diferentes zonas de la imagen.
- Realice un **análisis sobre la influencia del tamaño de la ventana** en los resultados obtenidos, utilizando diferentes tamaños de ventana.

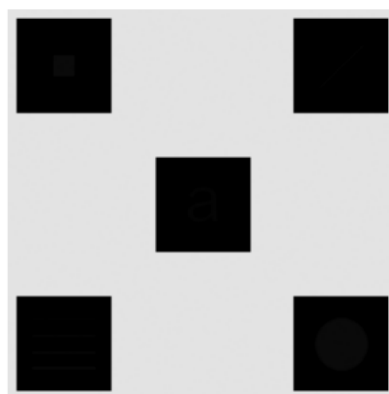


Figura 1: Imagen con detalles ocultos en diferentes zonas.

AYUDA: Con la función `cv2.copyMakeBorder(img, top, bottom, left, right, borderType)`, puede agregar una cantidad determinada de píxeles alrededor de una imagen. Los parámetros *top*, *bottom*, *left* y *right* indican la cantidad de píxeles que se agregarán en cada uno de los cuatro lados, mientras que *borderType* determina cómo se determinan los valores de los píxeles agregados. Por ejemplo, si se utiliza `borderType = cv2.BORDER_REPLICATE`, se replicarán los valores ubicados en los bordes.

Problema 2 - Validación de planilla de calificaciones

En la *figura 2* (archivo *grade_sheet_empty.png*) se muestra una **planilla de calificaciones**. Cada calificación (la cual ocupa una única fila de la planilla) posee un encabezado con dos campos para completar datos personales (*Legajo*, *Nombre* y *Apellido*) y cuatro campos para completar las calificaciones correspondientes (*Notas: Parcial 1, Parcial 2, Parcial 3, y Condición Final*).



Nro.	Legajo	Nombre y Apellido	Notas			Condición Final
			Parcial 1	Parcial 2	Parcial 3	
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						
19						
20						

Figura 2: Ejemplo del esquema de una planilla de calificaciones.

Se tiene una serie de planillas de calificaciones con registros (archivos *grade_sheet_<id>.png*), en formato imagen (.png), y se pretende **verificarlas de forma automática** por medio de un *script* en lenguaje *Python*. Para ello, debe considerar las siguientes restricciones de cada campo:

- Legajo:** Debe contener sólo 8 caracteres en total, formando una única palabra.
- Nombre y apellido:** Debe contener un mínimo de dos palabras y no más de 12 caracteres en total.
- Notas (Parcial 1, 2 y 3):** Cada campo debe contener 1 o 2 caracteres consecutivos (no deben existir espacios entre ellos).
- Condición Final:** Debe contener único caracter.

Asuma que todos los campos ocupan un sólo renglón y que sólo se utilizan los siguientes caracteres:

A B C D E F G H I J K L M N Ñ O P Q R S T U V W X Y Z 0 1 2 3 4 5 6 7 8 9 - /

AVISO: Sólo se debe verificar que cada dato cumpla con las restricciones de campo, NO se debe verificar si la información registrada es válida.



El algoritmo a desarrollar debe considerar y resolver los siguientes puntos:

- a. Se debe tomar únicamente como entrada la imagen de un formulario y mostrar por pantalla cuáles de los **campos** son **correctos (OK)** y cuáles son **incorrectos (MAL)**, por cada registro. Por ejemplo, en el terminal se debe visualizar:

```
> Registro 1:  
> Legajo: MAL  
> Nombre y apellido: OK  
> Parcial 1: OK  
> Parcial 2: MAL  
> Parcial 3: OK  
> Condición Final: OK  
>  
> Registro 2:  
> Legajo: OK  
> Nombre y apellido: OK  
...
```

En la *figura 3a* se muestra un ejemplo en donde los campos están todos correctamente completos, mientras que en la *figura 3b* se muestra otro ejemplo donde todos los campos están incorrectamente completos.

Nro.	Legajo	Nombre y Apellido	Notas			Condición Final
			Parcial 1	Parcial 2	Parcial 3	
1	C-1557/1	JUAN CARLINI	6	6	7	A

Figura 3a: Ejemplo de registro en donde todos los campos se cargaron correctamente.

Nro.	Legajo	Nombre y Apellido	Notas			Condición Final
			Parcial 1	Parcial 2	Parcial 3	
1	C-157/1	CARLINI		INS		Libre

Figura 3b: Ejemplo de registro en donde todos los campos se cargaron incorrectamente.

- b. Se debe generar una **única imagen de salida** que informe aquellos **alumnos que no hayan aprobado (Condición Final: "L" o "R")**, considerando solamente los registros que se hayan cargado correctamente. Dicha imagen de salida debe contar con el "crop" del contenido del campo *Nombre y Apellido* junto con algún indicador que diferencie a aquellos alumnos que deben recuperar (Condición Final: "R") de aquellos que se encuentran en condición de libre (Condición Final: "L").
- c. Se debe generar un **archivo CSV** para almacenar los resultados de cada validación. En el mismo, cada fila (registro) debe corresponder al conjunto de resultados de un registro de la planilla de calificaciones y debe responder a las siguientes pautas:
- Debe tener un identificador único (ID) en la primera columna cuyo valor se debe corresponder con el orden de los registros de la planilla de calificaciones.
 - Debe tener una serie de columnas correspondientes a los distintos campos de la planilla (respetando el orden): *Legajo*, *Nombre y Apellido*, *Parcial 1*, *Parcial 2*, *Parcial 3* y *Condición Final*.
 - Cada celda debe registrar el resultado de la validación, utilizando únicamente los valores posibles: *OK* o *MAL*.
- d. Se debe aplicar el algoritmo desarrollado, de **forma cíclica**, sobre el conjunto de cuatro imágenes de planillas con registros (archivos *grade_sheet_<id>.png*) e informar resultados.



AYUDAS:

- Existen varias formas de detectar las celdas donde se completan los datos, una de ellas es mediante la detección de las coordenadas de las líneas verticales y horizontales que alinean dichas celdas. Para ello, una opción posible consiste en umbralizar la imagen con **`img_th = img < th`** y luego sumar el valor de los píxeles que están en cada columna para detectar las líneas verticales con **`img_cols = np.sum(img_th_ones, 0)`** y sumar el valor de los píxeles que están en cada fila para detectar las líneas horizontales con **`img_rows = np.sum(img_th_ones, 1)`**. Luego, dado que en dichas líneas existen muchos más píxeles que en las demás partes del formulario, se puede definir un umbral acorde (uno para las líneas horizontales y otro para las líneas verticales) y de esta forma detectar las posiciones de las mismas. Por ejemplo, utilizando **`img_rows_th = img_rows > th_row`**. Tenga presente que las líneas pueden tener más de un píxel de ancho, por lo cual, quizás deba encontrar el principio y el fin de las mismas en **`img_rows_th`**.
- A partir de las sub-imágenes de los campos detectados, una posible forma de obtener cada caracter dentro de ellos, consiste en la obtención de las componentes conectadas con **`cv2.connectedComponentsWithStats(image, connectivity, ltype)`**. Sin embargo, tenga especial cuidado de que no hayan quedado píxeles de las líneas divisorias de la tabla dentro de dicha celda. Una posible forma de evitar este inconveniente, es con la eliminación de aquellas componentes conectadas que posean un área muy pequeña, mediante la definición de un umbral con **`ix_area = stats[:, -1] > th_area`** y luego aplicar un filtrado con **`stats = stats[ix_area, :]`**.